

halfedge：测地线距离模块设计文档

src/geodesics.rs

halfedge 库

2026-07-05

目录

1	模块定位	2
2	Heat Method 算法	3
2.1	Step 1: 时间步 $\tau = h^2$	3
2.2	Step 2: 热流扩散	4
2.3	Step 3: 单位负梯度	4
2.4	Step 4: 散度与 Poisson 求解	4
2.5	Step 5: 距离场偏移校正	4
3	Heat Method 算法流程图	5
4	离散算子	5
4.1	离散拉普拉斯（余切权重）	5
4.2	面梯度（FEM 形函数）	5
4.3	面散度（弱形式积分）	6
5	最短路径回溯	6
6	Dijkstra 最短图距离	6
6.1	最小堆实现	6
6.2	算法流程	7
6.3	多源 Dijkstra	7
6.4	路径回溯: dijkstra_with_parent	7
7	多源 Heat Method	8
8	MMP 精确测地线算法	8
8.1	核心概念：窗口（Window）	8
8.2	展开（Unfolding）	9
8.3	窗口传播	9
8.4	子窗口计算	9

8.5 算法流程图	11
8.6 多源 MMP: 统一传播	11
8.7 复杂度	12
9 三种算法对比	12
10 退化守卫	13
11 使用示例	13
12 测试覆盖	14

1 模块定位

`geodesics.rs` 在 `geometry.rs` / `linalg.rs` 之上实现三种测地线距离算法:

- **Heat Method** (Crane, Weischedel, Wardetzky 2013) —近似测地线
- **Dijkstra** —图上最短路径 (严格上界)
- **MMP** (Mitchell-Mount-Papadimitriou 1987) —精确测地线

并提供沿距离场梯度回溯最短路径的能力。

类别	API	语义
距离场 (Heat Method)	<code>geodesic_distance_from_vertex</code>	Heat Method: 从 <code>source</code> 出发计算所有顶点的测地线距离; 返回顺序对应 <code>mesh.vertex_ids()</code> ; 求解失败返回 <code>None</code> 。
路径回溯	<code>shortest_path</code>	沿距离场梯度回溯最短路径: 返回从 <code>target</code> 到源顶点的顶点序列 (含两端)。
距离场 (Dijkstra)	<code>dijkstra_geodesic</code>	单源 Dijkstra 最短图距离 (边长权值), 不依赖线性求解器
距离场 (Dijkstra 多源)	<code>dijkstra_multi_source_geodesic</code>	多源 Dijkstra: 每个顶点到最近源点的图距离
路径回溯 (Dijkstra)	<code>dijkstra_with_parent</code>	Dijkstra + 父节点回溯数组, 用于精确路径重建
路径 (Dijkstra 精确)	<code>dijkstra_shortest_path</code>	返回 <code>source</code> 到 <code>target</code> 的最短顶点序列 (含两端)
距离场 (多源 Heat Method)	<code>multi_source_geodesic</code>	多源 Heat Method: 每个顶点到最近源点的测地线距离; 源点强制为 0 消除数值误差
距离场 (MMP 精确)	<code>mmp_geodesic</code>	MMP 精确测地线: 从 <code>source</code> 出发计算所有顶点的精确测地线距离; 可穿过三角面内部, 非仅沿边
距离场 (MMP 多源)	<code>mmp_multi_source_geodesic</code>	多源 MMP: 每个顶点到最近源点的精确测地线距离

2 Heat Method 算法

Heat Method 把**非线性**的测地线距离计算转化为**两个线性系统**: 先用热流方程扩散初始脉冲得到光滑梯度方向, 再用 Poisson 方程从散度场恢复距离。其核心洞察是: 热流梯度的方向与测地线梯度方向几乎一致, 且方向对参数不敏感。

2.1 Step 1: 时间步 $\tau = h^2$

设网格平均边长为 h , 时间步取

$$\tau = h^2, \quad h = \frac{1}{|E|} \sum_{e \in E} L(e).$$

取 h^2 的理由: 使热流在每条边上的扩散量 $\tau \cdot \frac{1}{h^2} \sim O(1)$, 既不会过快抹平局部信息, 也不会过慢导致数值噪声。代码中若 $|E| = 0$, 回退 $\tau = 1$ 。

2.2 Step 2: 热流扩散

求解线性系统

$$(M + \tau L) u_\tau = M u_0,$$

其中 M 为 lumped mass (顶点 Voronoi 面积对角矩阵), L 为余切拉普拉斯, u_0 为初始热源。初始热源取源顶点 s 处的**单位积分脉冲**:

$$u_0[i] = \begin{cases} 1/M_{ss}, & i = s, \\ 0, & \text{otherwise.} \end{cases}$$

等价于热传导方程 $\frac{\partial u}{\partial t} = \Delta u$ 在时间 τ 后的解。代码使用**共轭梯度法** (conjugate_gradient) 求解, 最大迭代数 100 n , 相对残差 10^{-6} , 并预先调用 `regularize_diagonal` 加入 10^{-10} 对角正则化以保证 SPD 性。

2.3 Step 3: 单位负梯度

对每个三角面 f , 由分段线性基函数求 u_τ 的常梯度 $\nabla u_\tau|_f$, 再归一化并取负:

$$X_f = -\frac{\nabla u_\tau|_f}{\|\nabla u_\tau|_f\|}.$$

取负原因: 热流从高温(源)流向低温(远端), ∇u_τ 指向远离源, 而测地线最短路径应朝向源前进, 故取负。退化面 ($\|\nabla u_\tau\| < 10^{-10}$) 置 $X_f = \vec{0}$, 避免除零。

2.4 Step 4: 散度与 Poisson 求解

将单位负梯度场 X 投影回顶点散度 (弱形式):

$$(\nabla \cdot X)_i = \sum_{f \ni i} A_f \cdot (X_f \cdot \nabla \varphi_i|_f),$$

其中 φ_i 是顶点 i 的分片线性基函数。再求解 Poisson 方程

$$L \phi = \nabla \cdot X,$$

L 与 Step 2 共用同一份余切拉普拉斯矩阵 `cot_lap(CsMat<f64>)`: Step 1 通过 `build_laplacian_and_mass` 构建一次, Step 2 用其缩放 τL 加入 mass 对角得 $M + \tau L$, Step 4 直接 `clone` 后做对角正则化用于 Poisson 求解。**避免重复构建**拉普拉斯 (旧实现构建 3 次: Step 2、散度函数内部、Step 4 各一次)。同样使用共轭梯度法求解。

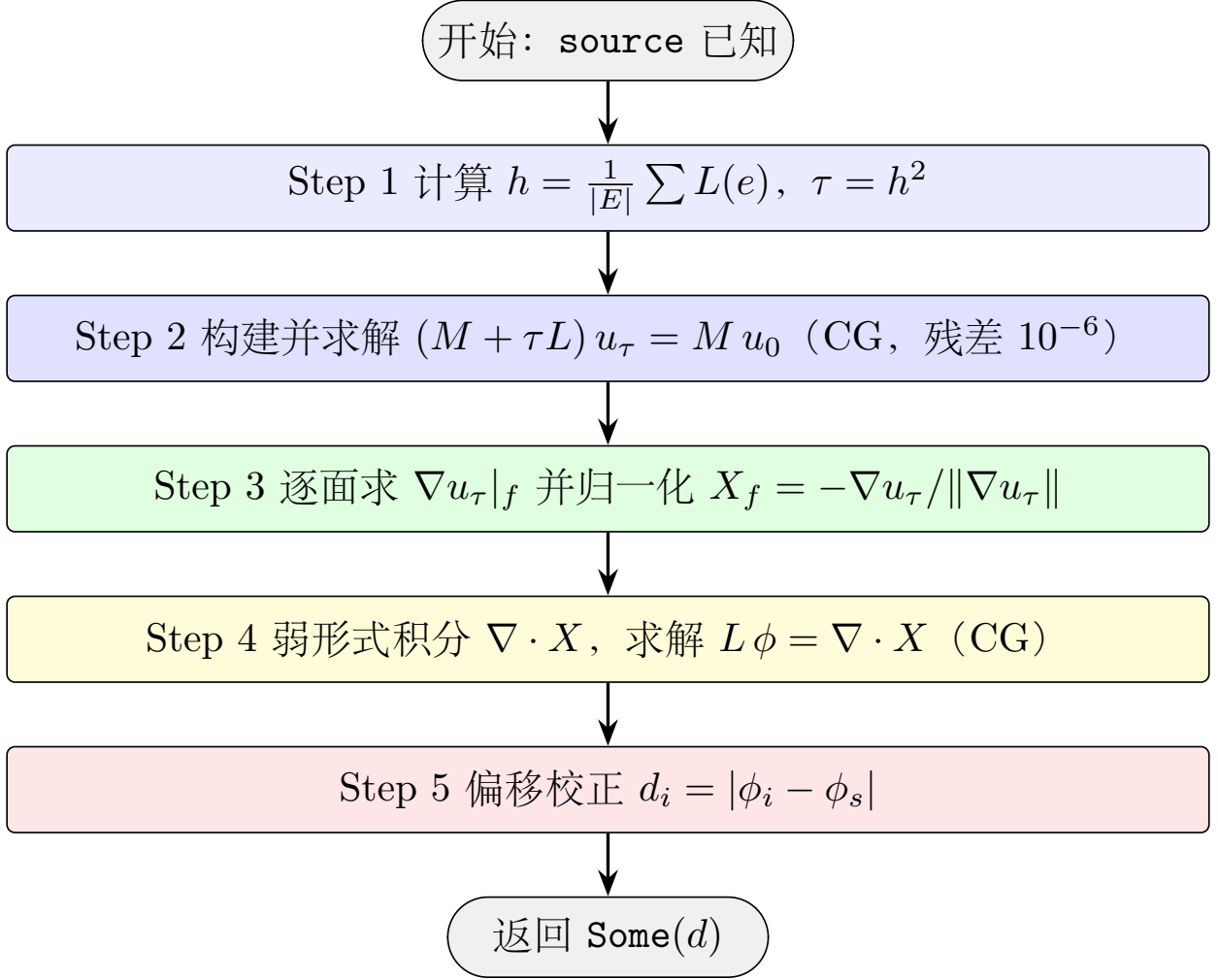
2.5 Step 5: 距离场偏移校正

Poisson 解 ϕ 与真实测地线距离 d 仅差一个常数 (Δ 的核空间)。取源顶点 s 处为零参考点:

$$d_i = |\phi_i - \phi_s|.$$

取绝对值是防止数值误差导致个别顶点 ϕ_i 略小于 ϕ_s 。校正后 $d_s = 0$, 其余 $d_i \geq 0$ 。

3 Heat Method 算法流程图



4 离散算子

4.1 离散拉普拉斯 (余切权重)

对相邻顶点 i, j , 设边 ij 的两个对角为 α_{ij}, β_{ij} , 则

$$L_{ij} = -\frac{1}{2}(\cot \alpha_{ij} + \cot \beta_{ij}), \quad L_{ii} = -\sum_{j \neq i} L_{ij}.$$

边界边只有一个对角, 使用单边余切。实现中仅累加 $w > 0$ 的项 (非钝角三角形贡献) 以避免负权重导致矩阵非 SPD。Lumped mass M_{ii} 取顶点周围面面积的 $1/3$ 之和, 退化时钳为 10^{-14} 。

4.2 面梯度 (FEM 形函数)

对三角面 f 的三顶点 i, j, k , 分片线性函数 $u|_f = u_i \varphi_i + u_j \varphi_j + u_k \varphi_k$ 的常梯度为

$$\nabla u|_f = \frac{1}{2A_f} \hat{n}_f \times (u_i \vec{e}_i + u_j \vec{e}_j + u_k \vec{e}_k),$$

其中 $\vec{e}_i = \vec{p}_k - \vec{p}_j$ 是顶点 i 所对的边向量, A_f 是面面积, $\hat{n}_f$ 是面法向。

4.3 面散度（弱形式积分）

向量场 X 在顶点 i 处的散度（弱形式）为

$$(\nabla \cdot X)_i = \sum_{f \ni i} A_f \cdot (X_f \cdot \nabla \varphi_i|_f), \quad \nabla \varphi_i|_f = \frac{1}{2A_f} \hat{n}_f \times \vec{e}_i,$$

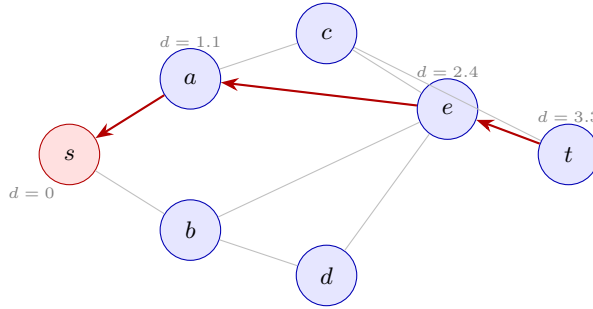
等价地写作

$$(\nabla \cdot X)_i = \frac{1}{2} \sum_{f \ni i} X_f \cdot (\hat{n}_f \times \vec{e}_i).$$

代码中 $\nabla \varphi_i|_f$ 直接用 $\hat{n}_f \times \vec{e}_i / (2A_f)$ 计算，再乘以 A_f 与 X_f 的点积，与上式一致。

5 最短路径回溯

`shortest_path` 在已计算好的距离场 d 上，沿**贪心梯度下降**方向回溯：当前顶点 v 的邻域中，选择 d 最小且严格小于当前距离的顶点 v' ，跳转至 v' ，重复直至到达源点（ $d < 10^{-10}$ ）或陷入局部极小（无更小邻点）。步数上界 $2|V|$ 防止死循环。



回溯方向： $t \rightarrow e \rightarrow a \rightarrow s$ （沿 d 严格递减）

贪心策略的局限：若距离场存在局部极小（数值误差或网格各向异性），可能在到达源点前提前终止；上界 $2|V|$ 保证不进入死循环。返回序列包含两端顶点，从 `target` 起到源点止。

6 Dijkstra 最短图距离

`dijkstra_geodesic` 使用经典 Dijkstra 算法计算**图最短距离**：以边长（欧氏长度）为权值，求源点到所有顶点的最短路径长度。相比 Heat Method，Dijkstra 路径只能沿网格边行进，结果为**图距离上界**，但实现简单、不依赖线性求解器，且结果**精确**（无数值误差）。

6.1 最小堆实现

Rust 标准库 `BinaryHeap` 是最大堆，通过反转 `Ord` 比较方向实现最小堆：

```
1 #[derive(Clone, Copy, Debug, PartialEq)]
2 struct QueueEntry { dist: f64, vertex: usize }
3
4 impl Ord for QueueEntry {
5     fn cmp(&self, other: &Self) -> Ordering {
6         // BinaryHeap
7         other.dist.partial_cmp(&self.dist).unwrap_or(Ordering::Equal)
```

```

8         .then_with(|| other.vertex.cmp(&self.vertex))
9     }
10 }

```

6.2 算法流程

1. 初始化 $\text{dist}[\text{source}] = 0$, 其余为 $+\infty$;
2. 将 $(0, \text{source})$ 入堆;
3. 循环弹出堆顶 (d, v) :
 - 若 $d > \text{dist}[v]$, 跳过 (已通过更短路径访问过);
 - 遍历 v 的邻居 u , 若 $d + \|p_u - p_v\| < \text{dist}[u]$, 更新 $\text{dist}[u]$ 并入堆 $(d + \|p_u - p_v\|, u)$;
4. 堆空时终止。

复杂度 $O(|E| \log |V|)$ 。

顶点索引反查的常数优化 主循环中需要把堆里存的索引 u 反查回 `VertexId`。早期实现调用 `mesh.vertex_ids().nth` 每次从迭代器头部推进 u 步, 单次 $O(u)$, 整体退化为 $O(|V|^2)$ 。修复方案: 入口处一次性构建 `vid_list: Vec<VertexId>` (与 `v_idx: HashMap<VertexId, usize>` 同步生成, 单次遍历), 循环内用 `vid_list[u]` 做 $O(1)$ 数组下标访问。`dijkstra_geodesic`、`dijkstra_multi_source_geodesic`、`dijkstra_with_parent` 三个入口均已采用此模式。

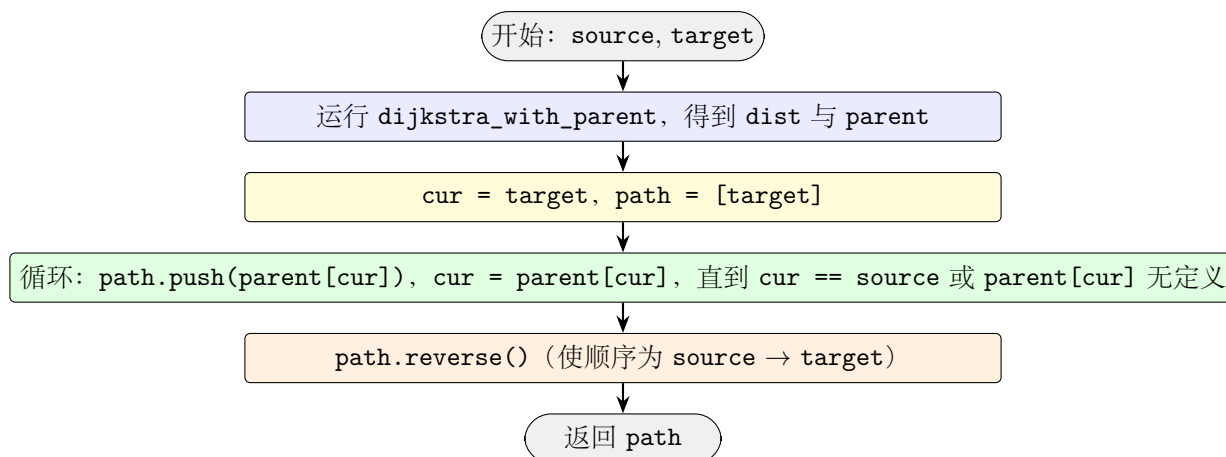
6.3 多源 Dijkstra

`dijkstra_multi_source_geodesic` 将所有源点以距离 0 同时入堆, 自然得到每个顶点到**最近源点**的距离。

6.4 路径回溯: `dijkstra_with_parent`

`dijkstra_with_parent` 在 Dijkstra 同时维护 `parent` 数组: 每次松弛 $\text{dist}[u]$ 时记录 $\text{parent}[u] = v$ 。返回 $(\text{dist}, \text{parent})$ 二元组, 便于精确路径重建。

`dijkstra_shortest_path` 在此基础上从 `target` 沿 `parent` 链回溯到 `source`, 得到精确最短顶点序列。



7 多源 Heat Method

`multi_source_geodesic` 将 Heat Method 扩展到**多源**场景：计算每个顶点到最近源点的测地线距离。实现上仅修改初始热源 u_0 ：

$$u_0[i] = \begin{cases} 1/M_{ii}, & i \in S(\text{任一源点}), \\ 0, & \text{otherwise.} \end{cases}$$

其余步骤(热流扩散、梯度归一化、Poisson 求解、偏移校正)与单源完全相同,且**共用同一份** `cot_lap`: 构建一次后热流系统与 Poisson 系统分别引用 (Poisson 求 `clone` 后做正则化), 避免重复构建。

源点数值误差处理: Heat Method 在源点处存在数值误差, 单源时通过 ϕ_s 偏移校正消除; 多源时各源点误差不一致, 故在偏移校正后**显式将每个源点距离置零**:

$$\forall s \in S: d_s \leftarrow 0.$$

8 MMP 精确测地线算法

`mmp_geodesic` 实现 Mitchell-Mount-Papadimitriou (1987) 精确测地线算法, 通过**窗口传播**(window propagation) 在三角面上追踪伪波前, 计算从源顶点到所有顶点的精确测地线距离。与 Dijkstra 不同, MMP 的测地线路径可**穿过三角面内部**, 而非仅沿边行进; 与 Heat Method 不同, MMP 给出**精确**距离而非近似值。

8.1 核心概念: 窗口 (Window)

窗口是三角面边上的一个**伪波前段**, 记录了波前到达边上一个区间的信息:

字段	语义
<code>he</code>	窗口所在的半边
<code>b0, b1</code>	窗口在边上的参数范围 $[b_0, b_1] \subset [0, 1]$; $0 = \text{origin}(\text{he.twin.vertex})$, $1 = \text{tip}(\text{he.vertex})$
<code>d0, d1</code>	b_0 和 b_1 处的测地线距离
<code>pseudo_src</code>	伪源位置 (3D), 经展开反射后; 窗口内任意点 x 的距离为 $\ S' - x\ $
<code>from_face</code>	窗口来自的面 (传播方向: 进入另一个面)

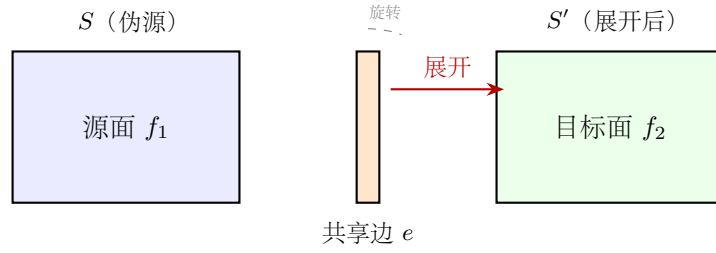
窗口内任意点 x 的测地线距离等于伪源到该点的欧氏距离:

$$d(x) = \|S' - x\|.$$

这一性质是 MMP 算法正确性的基础: 展开操作保证从原始源到 x 的测地线路径在展开后成为直线段。

8.2 展开 (Unfolding)

当窗口跨过共享边从源面传播到目标面时，需要将伪源从源面平面“展开”到目标面平面。展开操作保持伪源到共享边上各点距离不变，同时使伪源到目标面内各点的 3D 距离等于测地线距离。



实现中，展开通过构建局部坐标系完成：

1. 以共享边为 x 轴构建局部坐标系；
2. 在源面坐标系中表达伪源坐标 (s_x, s_y) ；
3. 将坐标映射到目标面坐标系（更换 y 轴方向为目标面法向侧）。

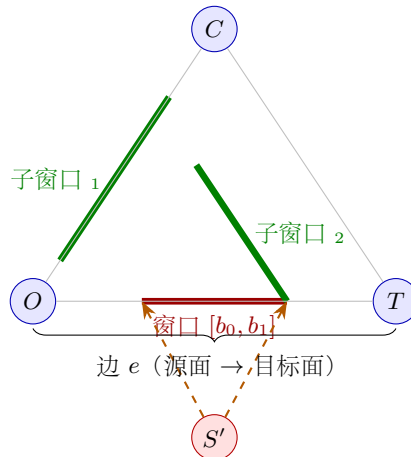
8.3 窗口传播

给定窗口 w 位于边 e 上， $\text{from_face} = f_1$ ，传播步骤如下：

1. 确定目标面： f_2 为边 e 的另一个邻接面 ($\neq f_1$)；
2. 获取对面顶点： f_2 的三个顶点中不在 e 上的那个顶点 C ；
3. 展开伪源： $S' = \text{unfold}(S, f_1, f_2, e)$ ；
4. 更新顶点距离： $d_C = \min(d_C, \|S' - C\|)$ ；
5. 计算子窗口：伪源 S' 通过窗口 $[b_0, b_1]$ 发出的射线楔与两条子边 ($O \rightarrow C$ 和 $T \rightarrow C$) 的交集。

8.4 子窗口计算

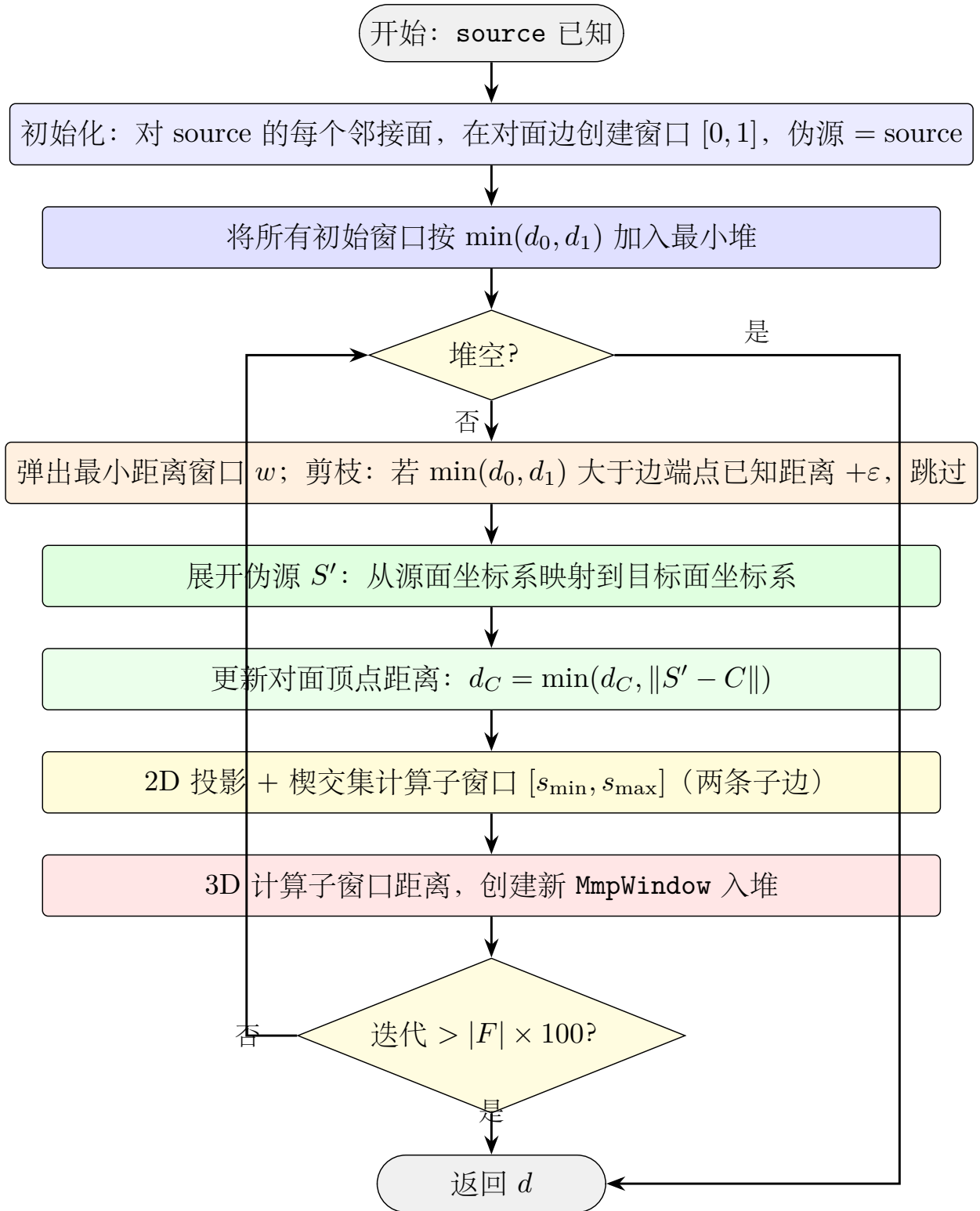
子窗口的计算在 2D 局部坐标系中进行（投影到面平面后保持等距性）：



具体步骤:

1. 将伪源 S' 、窗口端点 $P(b_0)/P(b_1)$ 、子边端点投影到面平面 2D 坐标系;
2. 构建照明楔: 从 S' 出发通过 $P(b_0)$ 和 $P(b_1)$ 的两条射线;
3. 对每条子边, 计算楔与子边的交集:
 - 检查子边端点是否在楔内 (2D 叉积判定);
 - 计算楔边界射线与子边的交点 (2D 射线-线段求交);
 - 合并所有交点得到参数区间 $[s_{\min}, s_{\max}]$;
4. 在 3D 中计算子窗口端点的距离: $d = \|S' - Q\|$ (使用展开后的伪源);
5. 创建新的 `MmpWindow` 加入优先队列。

8.5 算法流程图



8.6 多源 MMP: 统一传播

`mmp_multi_source_geodesic` 与单源 `mmp_geodesic` 共用同一核心 `mmp_geodesic_impl`。多源扩展的关键是**初始化阶段**:

1. 遍历所有源 $s \in S$, 对每个 s 的邻接面创建初始窗口;

2. 所有窗口放入同一个优先队列；
3. 主循环与单源完全相同——窗口传播自然处理来自不同源的波前相遇。

为何不逐源运行取最小？ 旧实现「for s in S: d_s = mmp(s); result = min(result, d_s)」复杂度为 $O(S \cdot n^2 \log n)$ ，随源数 S 线性增长。统一传播复杂度仍为 $O(n^2 \log n)$ （窗口总数随 S 略增但同阶），不随 S 线性退化。

正确性 统一传播得到的距离场 $d_{\text{unified}}[v]$ 与逐源取最小 $d_{\text{naive}}[v] = \min_{s \in S} d_s[v]$ 在浮点容差内数值一致（测试 `mmp_multi_source_unified_matches_naive` 验证）。原因：MMP 窗口的伪源位置 `pseudo_src` 已编码源信息，不同源的窗口在传播时互不干扰，相遇时距离较小者自然胜出。

8.7 复杂度

最坏情况 $O(n^2 \log n)$ ，实践中通常远好于此。剪枝策略（跳过最小距离大于已知顶点距离的窗口）有效减少了实际处理的窗口数量。

9 三种算法对比

属性	Heat Method	Dijkstra	MMP
路径类型	测地线（可跨面）	图路径（仅沿边）	测地线（可跨面）
精度	近似（数值求解）	图精确（无误差）	精确（无离散化误差）
依赖	稀疏线性求解器（CG）	仅需最小堆	仅需最小堆
复杂度	$O(k \cdot n)$ （CG 迭代）	$O(E \log V)$	$O(n^2 \log n)$ 最坏
源点误差	有（偏移校正后 ≈ 0 ）	无	无
多源扩展	修改 u_0 + 显式置零	多源同时入堆	多源同时入堆（统一传播）
路径提取	梯度回溯（近似）	parent 回溯（精确图路径）	梯度回溯（近似）

关系：

- Dijkstra 距离是真实测地线距离的**上界**（仅沿边，路径更长）；
- MMP 距离是真实测地线距离的**精确值**（可穿面）；
- Heat Method 距离是真实测地线距离的**近似值**（数值求解）；
- 因此 $d_{\text{MMP}} \leq d_{\text{Dijkstra}}$ 恒成立。

10 退化守卫

函数	退化场景	处理
<code>geodesic_distance_from_vertex</code>	空网格 ($ V = 0$)	返回 <code>Some(Vec::new())</code>
<code>geodesic_distance_from_vertex</code>	<code>source</code> 不在 <code>v_idx</code> 中	返回 <code>None</code> (? 传播)
<code>geodesic_distance_from_vertex</code>	$ E = 0$	时间步回退 $\tau = 1$
<code>geodesic_distance_from_vertex</code>	共轭梯度未收敛	CG 返回 <code>None</code> , 函数返回 <code>None</code>
<code>build_laplacian_and_mass</code>	顶点面积 $< 10^{-14}$	钳为 10^{-14} , 防除零
<code>build_laplacian_and_mass</code>	余切权重 $w \leq 0$	跳过该边, 避免破坏 SPD
<code>compute_face_gradients</code>	面积 $< 10^{-14}$ (退化面)	跳过该面
Step 3 归一化	$\ \nabla u_\tau\ < 10^{-10}$	$X_f = \vec{0}$
<code>shortest_path</code>	<code>target</code> 无效或越界	返回 <code>Vec::new()</code>
<code>shortest_path</code>	陷入局部极小	提前终止, 返回已收集路径
<code>shortest_path</code>	步数达上界 $2 V $	终止循环, 防止死循环
<code>mmp_geodesic</code>	空网格	返回空 <code>Vec</code>
<code>mmp_geodesic</code>	<code>source</code> 无效	返回全 <code>INFINITY</code>
<code>mmp_geodesic</code>	面非三角 ($ he \neq 3$)	跳过该面
<code>mmp_geodesic</code>	迭代超限 ($> F \times 100$)	终止循环
<code>unfold_pseudo_source</code>	退化边 (长度 $< 10^{-14}$)	返回原始伪源 (不反射)
<code>propagate_window</code>	边界半边 (无 <code>twin</code>)	返回空窗口列表
<code>propagate_window</code>	子边长度 $< 10^{-14}$	跳过该子边
<code>propagate_window</code>	窗口区间 $< 10^{-10}$	跳过 (退化窗口)

11 使用示例

```

1 use halfedge::geodesics::{
2     geodesic_distance_from_vertex, shortest_path,
3     dijkstra_geodesic, mmp_geodesic,
4 };
5 use halfedge::test_util::build_icosphere;
6
7 let mesh = build_icosphere(2);
8 let vertices: Vec<_> = mesh.vertex_ids().collect();
9 let source = vertices[0];
10 let target = vertices[vertices.len() / 2];
11
12 // 1. Heat Method
13 let dist_heat = geodesic_distance_from_vertex(&mesh, source)
14     .expect("heat method ok");
15

```

```

16 // 2. Dijkstra
17 let dist_dijk = dijkstra_geodesic(&mesh, source);
18
19 // 3. MMP
20 let dist_mmp = mmp_geodesic(&mesh, source);
21
22 //          <=
23 assert!(dist_mmp[target_idx] <= dist_dijk[target_idx] + 1e-8);
24
25 // 4. target
26 let path = shortest_path(&mesh, &dist_mmp, target);

```

12 测试覆盖

测试名	验证点
test_geodesic_self_distance	在 icosphere 上调用 <code>geodesic_distance_from_vertex</code> , 断言返回 <code>Some</code> ; 源顶点距离 $< 10^{-6}$; 至少一个非源顶点距离 > 0 。
test_geodesic_monotonicity	在 icosphere 上对前若干非源顶点做 sanity check: 领域中至少存在一个顶点距离更小, 验证距离场单调性。
test_dijkstra_self_distance_zero	源点到自身 Dijkstra 距离 = 0
test_dijkstra_symmetric_on_icosphere	icosphere 上 $d(s \rightarrow t) \approx d(t \rightarrow s)$ (容差 10^{-6})
test_dijkstra_distance_upper_bounds_heat_method	Dijkstra 与 Heat Method 的最大距离在 3 倍以内 (图距离 vs 测地线)
test_dijkstra_multi_source	多源 Dijkstra: 每个顶点距离 $\leq$ 单源距离; 源点距离 = 0
test_dijkstra_shortest_path_self	<code>source == target</code> 时返回 <code>[source]</code>
test_dijkstra_shortest_path_to_neighbor	邻居路径长度为 2 (<code>[source, target]</code>)
test_dijkstra_shortest_path_consistency_with_distance	路径长度与距离一致: $\text{dist}[\text{target}] \approx \sum \ p_{i+1} - p_i\ $
test_multi_source_geodesic_sources_zero	多源 Heat Method 所有源点距离 $< 10^{-6}$ (强制置零生效)
test_multi_source_geodesic_le_single_source	多源最大距离 $\leq$ 单源最大距离 (容差 10^{-6})
mmp_self_distance_zero	MMP 源顶点距离 = 0
mmp_le_dijkstra	MMP 距离 $\leq$ Dijkstra 距离 (可穿面, 路径更短)
mmp_symmetric	icosphere 上 $d(s \rightarrow t) \approx d(t \rightarrow s)$ (5% 容差)
mmp_multi_source	多源 MMP 距离 $\leq$ 单源距离
mmp_multi_source_unified_matches_naive	统一传播与逐源取最小数值一致 (容差 10^{-9})
mmp_flat_grid	平面网格上 MMP 距离接近欧氏距离 (容差 0.1-0.2)

接下页

(续表)

测试名	验证点
边界与无效输入测试 (<i>icosphere(0)</i> 与空网格)	
<code>dijkstra_empty_mesh_returns_empty</code>	空网格 <code>MeshStorage::new()</code> 上 <code>dijkstra_geodesic</code> 返回空 <code>Vec</code>
<code>mmp_empty_mesh_returns_empty</code>	空网格上 <code>mmp_geodesic</code> 返回空 <code>Vec</code>
<code>dijkstra_multi_source_empty_sources_returns_infinity</code>	非空网格但 <code>sources = &[]</code> 时返回全 <code>INFINITY</code>
<code>dijkstra_invalid_source_returns_infinity</code>	<code>VertexId::default()</code> 不在网格中, 返回全 <code>INFINITY</code>
<code>mmp_invalid_source_returns_infinity</code>	同上用 <code>mmp_geodesic</code> , 返回全 <code>INFINITY</code>
<code>geodesic_distance_from_vertex_invalid_source_returns_none</code>	无效 <code>source</code> 经 ? 传播返回 <code>None</code>
<code>multi_source_geodesic_empty_sources_returns_none</code>	空源集直接返回 <code>None</code>
<code>dijkstra_with_parent_invalid_source_returns_empty</code>	无效 <code>source</code> 返回 (<code>Vec::new()</code> , <code>Vec::new()</code>)
<code>dijkstra_shortest_path_invalid_target_returns_empty</code>	无效 <code>target</code> 返回空 <code>Vec</code>
<code>dijkstra_distance_nonnegative</code>	所有 Dijkstra 距离 ≥ 0
<code>mmp_multi_source_single_source_matches_naive</code>	<code>mmp_multi_source_geodesic(&[v0])</code> 与 <code>mmp_geodesic(v0)</code> 数值一致
<code>mmp_distance_nonnegative</code>	所有 MMP 距离 ≥ 0
<code>shortest_path_to_self_returns_self</code>	<code>target == source</code> 时回溯立即终止, 路径 = <code>[source]</code>
<code>shortest_path_invalid_distance_returns_empty</code>	空距离场 <code>&[]</code> 时 <code>target_idx >= len</code> 返回空 <code>Vec</code>

`src/geodesics.rs` 共 31 个单元测试, 全库共 567 个。